

# A Tensor Network-based Quantum Algorithm for the Nonlinear 1D Burgers' Equation

Muralikrishnan Gopalakrishnan Meena\*

*Oak Ridge National Laboratory, Oak Ridge, TN, 37830, USA*

Vincent Jones<sup>†</sup>

*University of Virginia, Charlottesville, VA, 22904, USA*

Yu Zhang<sup>‡</sup>

*Los Alamos National Laboratory, Los Alamos, NM, 87545, USA*

Xinfeng Gao<sup>§</sup>

*University of Virginia, Charlottesville, VA, 22904, USA*

In this work, we implement a tensor network-based quantum algorithm to solve unsteady, nonlinear partial differential equations (PDEs). The challenge lies in how to effectively represent, encode, process, and evolve the nonlinear system of PDEs on quantum computers. We will discuss the new techniques using the compressible 1-dimensional (1D) Burgers' equation as an example, because it represents the fundamental nonlinear feature and yet removes certain complexity in physics, allowing us to focus on the design of quantum algorithms. Previous attempts to solve nonlinear PDEs in quantum computation have often involved storing multiple copies of solutions or employing linearizations. Neither is practical due to exponential scaling with evolution time or insufficient solution accuracy. Our framework is based on matrix product states (MPSs) and matrix product operators (MPOs). For example, the velocity field is represented by MPS, whereas the linear and nonlinear spatial differential terms of the velocity field are processed by MPOs. Our primary focus herein is to verify and validate the various tensor network components of the algorithm using solutions obtained by the classical algorithms on high performance computing (HPC) architectures. We use a classical time marching method to demonstrate the functionality of the tensor network operations to model the PDE and their robustness with the time evolution of the system. Our classical simulation results demonstrate the utility of tensor network-based operations in modeling nonlinear PDEs and highlight the necessity as well as potential advantages of using quantum simulations for these techniques. \*

## I. Nomenclature

|               |   |                                              |
|---------------|---|----------------------------------------------|
| $\mathbf{u}$  | = | velocity field in $\mathcal{D}$ dimensions   |
| $\mathcal{D}$ | = | number of spatial dimensions                 |
| $\mathbf{x}$  | = | spatial location in $\mathcal{D}$ dimensions |
| $u$           | = | velocity in one dimension                    |
| $u_0$         | = | velocity at initial condition                |
| $\nu$         | = | kinematic viscosity                          |
| $\beta$       | = | coefficient of compressibility               |

---

\*Computational Scientist, National Center for Computational Sciences, gopalakrishnm@ornl.gov.

<sup>†</sup>Undergraduate Research Assistant, CFD & Propulsion Laboratory, Department of Mechanical and Aerospace Engineering, yna7te@virginia.edu.

<sup>‡</sup>Staff Scientist, Theoretical Division, zhy@lanl.gov.

<sup>§</sup>Professor, CFD & Propulsion Laboratory, Department of Mechanical and Aerospace Engineering, gao@virginia.edu.

\*This manuscript has been authored by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript or allow others to do so for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<http://energy.gov/downloads/doe-public-access-plan>).

|                  |   |                                                                        |
|------------------|---|------------------------------------------------------------------------|
| $g$              | = | forcing term                                                           |
| $N$              | = | number of grid points used to discretize the spatial domain            |
| $q$              | = | number of quantum registers or qubits                                  |
| $K$              | = | the quantum register values. $K = 2$ for qubits.                       |
| $A$              | = | a third-order tensor                                                   |
| $d$              | = | bond dimension for tensor decomposition                                |
| $ u\rangle$      | = | quantum state representation of classical variable $u$                 |
| $S^\pm$          | = | ladder operator                                                        |
| $\delta x$       | = | spatial step                                                           |
| $\delta \tau$    | = | temporal step                                                          |
| $O^{\nabla^n} u$ | = | $n$ -th order spatial differential operator on $u$                     |
| $\hat{H}$        | = | Hamiltonian operator                                                   |
| $M$              | = | number of time steps                                                   |
| $\hat{A}$        | = | a Hermitian matrix made up of a linear combination of unitary matrices |
| $\hat{P}_i$      | = | unitary Pauli matrices                                                 |
| $c$              | = | coefficient vector for the unitary decomposition of $\hat{A}$          |

## II. Introduction

PARTIAL differential equations (PDEs) are fundamental to modeling physical systems across a wide range of disciplines, including climate science, biology, astrophysics, aerospace engineering, and national security. The primary challenge in solving these equations numerically lies in the presence of nonlinear terms. In fluid dynamics, for example, the existence and smoothness of solutions to the full three-dimensional (3D) Navier-Stokes equations remain analytically unresolved. Numerical simulations offer valuable insight into their behavior and the potential formation of singularities or the stability of solutions. Despite major advances in classical high-performance computing (HPC), solving the 3D Navier-Stokes equations with realistic turbulent physics remains computationally demanding, especially for high-Reynolds-number flows. Even with exascale systems, such simulations are resource-intensive [1]. For example, in 2023, turbulence-related projects accounted for approximately 35 – 45% of total usage on the Oak Ridge Leadership Computing Facility’s Frontier supercomputer.

Quantum computing has the potential to offer a new paradigm for solving these computationally intractable problems with classical HPC. Our overarching goal is to develop efficient quantum algorithms for solving large nonlinear systems of PDEs, particularly those governing turbulent fluid dynamics. However, quantum algorithms capable of efficiently solving nonlinear PDEs remain limited [2]. Many current strategies rely on linearizing the nonlinear system [3–7] and then applying quantum linear system solvers (QLSA) [8, 9] to the resulting ODEs. Examples include Carleman linearization for quadratic nonlinear systems [10, 11], and Koopman-von Neumann approaches for representing nonlinear dynamics in a linear operator framework [12–14]. Recent developments have begun to address nonlinear PDEs more directly [2, 15], including techniques based on quantum amplitude estimation [16, 17]. Other promising directions involve variational quantum algorithms and tensor networks for handling nonlinearities in single-variable PDEs [18, 19]. The quantum nonlinear processing unit (QNPU) is a novel architecture for treating such nonlinearities, but its application to systems of PDEs, such as the full Navier-Stokes equations, introduces significant additional complexity. Krylov subspace methods and circuit-based tensor operations may offer alternatives to variational optimization, though they pose measurement and scaling challenges due to the complexity of the resulting operator expressions.

In this paper, we provide a preliminary demonstration of the quantum algorithm framework introduced in [20], which is based on tensor networks, using the Burgers’ equation to illustrate the main concept. That is, how nonlinear terms in the PDE can be effectively processed using tensor network-based algorithms. The Burgers’ equation is a fundamental PDE representing both convection and diffusion processes, without any additional complexities imparted by the full compressible Navier-Stokes equations for various high-speed flow applications. Thus, it provides a great avenue to validate and demonstrate the capability of the quantum algorithm, which is also relevant to various practical fluid dynamics applications.

In what follows, we first elaborate on the technical approach, then detail the implementation process of the quantum algorithm, highlighting the verification and validation procedures of the tensor network algorithm, and discuss the results by focusing on the performance of the tensor network operators. Finally, insights from this study as well as future extensions of the algorithm are provided.

### III. Technical Approach

#### A. Governing equation: The Burgers' equation

The Burgers' equation serves as a good test case for the development of new quantum algorithms for fluid dynamics problems due to its simplified yet representative feature in mathematical properties, the ability to model challenging dynamics such as shock and discontinuities, and the availability of exact solution for verification. The current study focuses on the 1D form, making the comparison between the classical and quantum solutions straightforward.

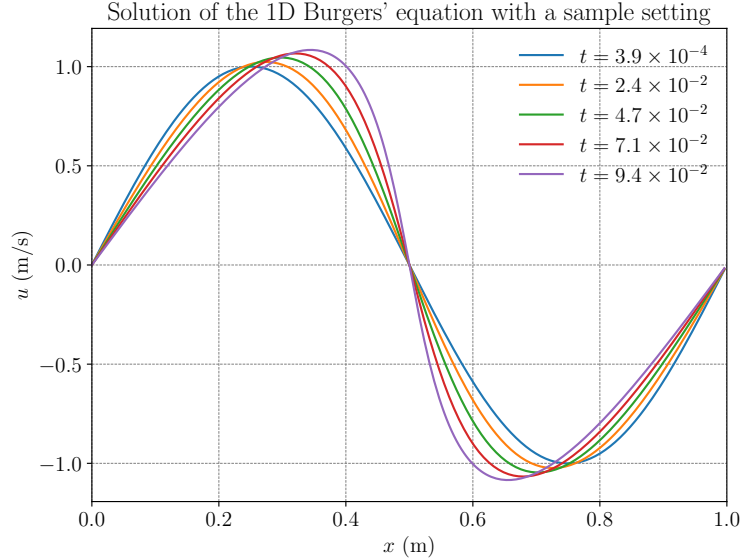
The three-dimensional compressible Burgers' equation is given by

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} - \nu \nabla^2 \mathbf{u} = \beta \nabla (\nabla \cdot \mathbf{u}), \quad (1)$$

where,  $\mathbf{u} = (u, v, w)$  is the velocity field,  $\nu$  is the kinematic viscosity, and  $\beta$  is a coefficient related to compressibility. The source term,  $\beta \nabla (\nabla \cdot \mathbf{u})$ , accounts for the effects of compressibility, capturing how local changes in fluid volume influence the overall flow dynamics. The 1D configuration is given by

$$\frac{\partial u}{\partial t} + \frac{1}{2} \frac{\partial uu}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2} = g(x, t), \quad (2)$$

where  $u$  is the instantaneous velocity and  $g(x, t)$  is a forcing term. Within this study, a spatially-temporally varying source term is specified for the viscous Burgers' equation. This forcing term provides an additional assessment of the tensor network algorithm. A sample solution of the Burgers' equation is provided in Fig. 1, where Eq. 2 is solved (classically) using a finite difference method - forward time-central space (FTCS) method. For this sample solution, we use a kinematic viscosity of  $\nu = 1 \times 10^{-4} \text{ m}^2/\text{s}$ , discretized on  $N = 200$  grid points in the computational domain of  $x \in [0, 1]$ , set the source term as  $g = \sin(2\pi x) \cos(2\pi t)$ , and the initial condition as  $u(x, t = 0) = u_0 = \sin(2\pi x)$ .



**Fig. 1** A sample time evolution of the Burgers' equation with  $\nu = 1 \times 10^{-4} \text{ m}^2/\text{s}$ ,  $g = \sin(2\pi x) \cos(2\pi t)$ , and initial condition  $u_0 = \sin(2\pi x)$ .

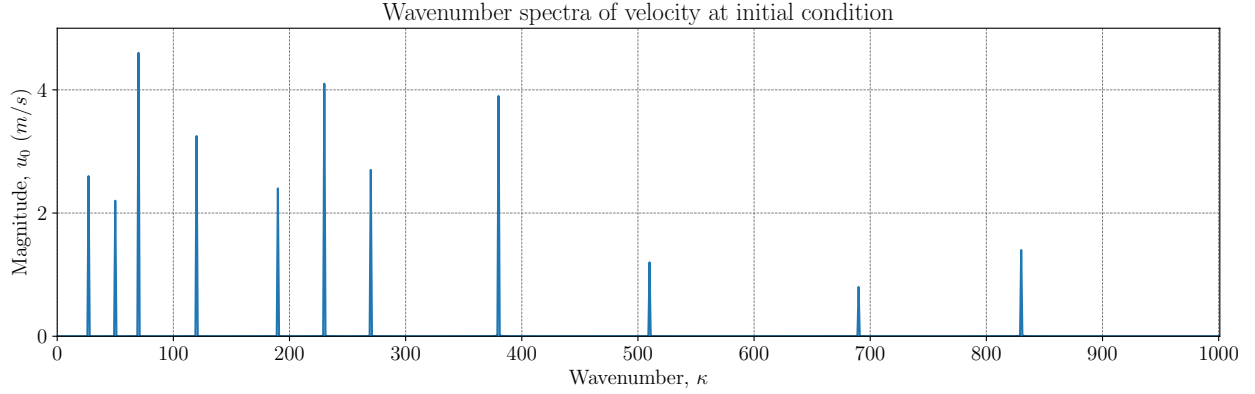
For demonstrating the capability of the quantum algorithm, we also choose to solve a much more complex

phenomenon for which the forcing term,  $g(x, t)$ , and the initial condition,  $u_0$ , are given by

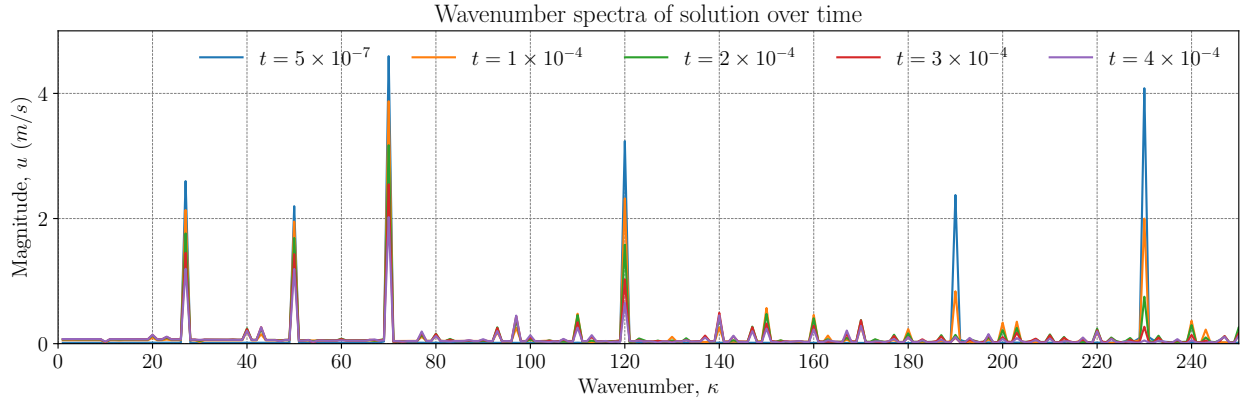
$$g(x, t) = 2 + \frac{19}{5} \sin(46\pi x + 74\pi t) + \frac{8}{5} \sin(140\pi x + 20\pi t + \frac{1}{4}) + \frac{24}{5} \sin(120\pi x + 22\pi t) \sin(204\pi x + 60\pi t), \quad (3)$$

$$u_0 = 30 + \frac{13}{5} \sin(54\pi x) + \frac{11}{5} \sin(100\pi x + 0.2) + \frac{23}{5} \sin(140\pi x + 0.05) + \frac{13}{4} \sin(240\pi x + 0.6) + \frac{12}{5} \sin(380\pi x + 1.2) + \frac{41}{10} \sin(460\pi x + 0.3) + \frac{27}{10} \sin(540\pi x + 0.9) + \frac{39}{10} \sin(760\pi x + 0.5) + \frac{6}{5} \sin(1020\pi x + 0.1) + \frac{4}{5} \sin(1380\pi x + 2) + \frac{7}{5} \sin(1660\pi x + 1.7). \quad (4)$$

The wavenumber spectra of  $u_0$  are shown in Fig. 2. The kinematic viscosity is  $1 \times 10^{-5} \text{ m}^2/\text{s}$ . For the computational domain,  $x \in [0, 1]$ , the classical HPC solution for a computational grid of  $N = 100,000$  cells is referred to as the “DNS” solution [21]. A sample depiction of the time evolution of the flow is shown in Fig. 3.



**Fig. 2 Wavenumber spectra of the initial condition of the flow solved in Walters et al..**



**Fig. 3 Time evolution of the flow solved in Walters et al. depicted using wavenumber spectra.**

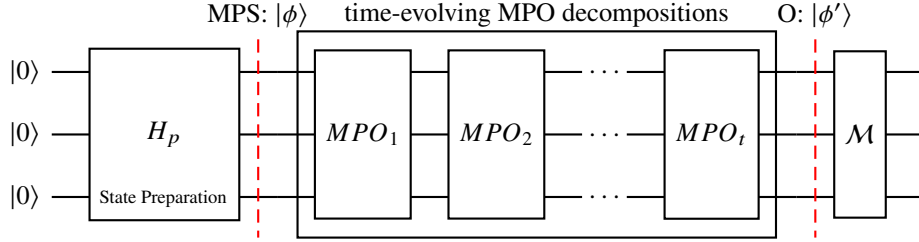
Next, we describe the quantum algorithm for evaluating the linear and nonlinear terms in the Burgers’ equation and for time evolving the PDE. We note that the quantum algorithm discussed in this work can be extended to evaluate the linear and nonlinear terms in the momentum and pressure Poisson equations of the Navier-Stokes equations and for evolving the coupled system of PDEs. In this preliminary work, we focus on the 1D Burgers’ equation to exhaustively describe the underlying quantum algorithm.

## B. Quantum algorithm

The main idea behind the algorithm is to represent the classical solution variable(s) (i.e.,  $\mathbf{u}$  for Burgers' equation) in quantum states by transferring Eq. 2 to a Hamiltonian evolution problem. There are three steps in this algorithm:

- 1) Quantum embedding of  $u$  - use matrix product state (MPS)
- 2) Quantum representation of the linear and nonlinear differential operations - use matrix product operator (MPO)
- 3) Time evolution of the quantum operations - use fast-forwarding algorithms (*proposed*)

Fig. 4 illustrates the steps from left to right. That is, prepare the initial state and the initial Hamiltonian, encode the variable states in MPSs, advance the states by MPOs, measure, and analyze. In the following sections, we discuss the technical details.



**Fig. 4** Sample depiction of the tensor network-based quantum circuit representing the time evolution of the nonlinear equations.

### 1. Quantum embedding of velocity using MPS

One of the most challenging aspects of translating quantum algorithms to solve classical problems in various domain applications is embedding the state information into quantum states. Inspired by tensor networks, we embed the flow states as matrix product states (MPSs) [22, 23].

The key idea is to decompose each component of the  $\mathcal{D}$ -dimensional velocity variable  $\mathbf{u} = \sum_{\alpha}^{\mathcal{D}} u_{\alpha}(\mathbf{x}) \mathbf{e}_{\alpha}$  into a series of low-rank tensors. For Eq. 2, we have  $\mathcal{D} = 1$ . Suppose that the PDE is spatially discretized on a grid with  $N = K^q$  grid points in each dimension, then the total number of grid points is  $K^{\mathcal{D}q}$ . Herein,  $N$  is the classical registers (i.e., the mesh size),  $q$  the number of quantum registers (i.e., the number of qubits),  $K$  the quantum states of a quantum register ( $K = 2$  for qubits), and  $\mathcal{D}q$  is the total number of qubits, simply  $\mathcal{D} \times q$ . Thus, the velocity variable is approximated by  $u_{\alpha}(\mathbf{x}) \simeq u_{\alpha}(x_1, \dots, x_{\mathcal{D}q}) \equiv u_{\alpha}(\{x_i\})$ . Here,  $i = 1, 2, \dots, \mathcal{D}q$  and  $x_i$  denote the position of the element in the mapped quantum state space, and its value ranges from 0 to  $K - 1$ . Then we can decompose the function  $u_{\alpha}(\{x_i\})$  into matrix product state (MPS) via the Schmidt decomposition,

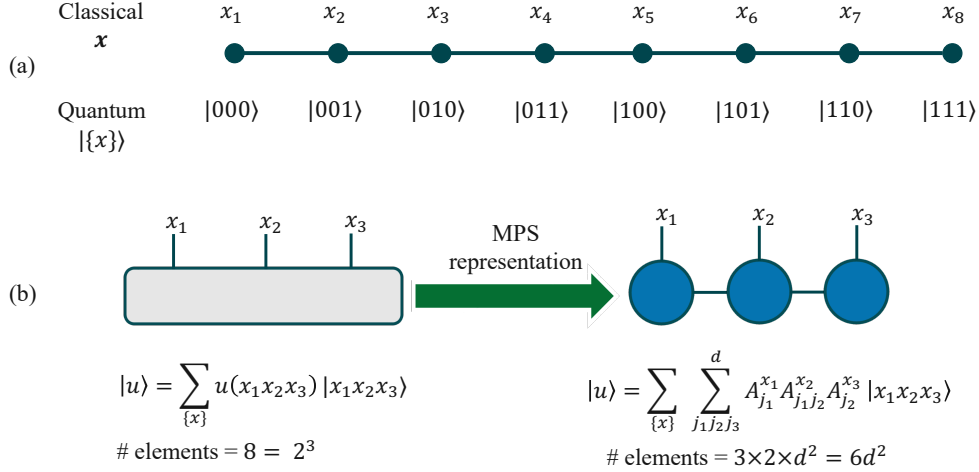
$$u_{\alpha}(\mathbf{x}) = \sum_{j_1, \dots, j_{\mathcal{D}q-1}}^d A_{j_1}^1 A_{j_1, j_2}^2 \cdots A_{j_{\mathcal{D}q-1}}^{\mathcal{D}q}, \quad (5)$$

where  $d$  is the bond dimension (or rank) that controls the quality of the tensor decomposition and  $A^k$  is a third-order tensor. Herein, we only consider the algorithm for the 1D Burgers' equation, which gives the MPS decomposition of velocity in Eq. 2 as

$$u(x) = \sum_{j_1, \dots, j_{q-1}}^d A_{j_1}^1 A_{j_1, j_2}^2 \cdots A_{j_{q-1}}^q, \quad (6)$$

where velocity  $u(x) \equiv u(\{x_i\})$  for  $i = 1, 2, \dots, q$ . A sample representation of a 1D flow field on  $N = 8$  grid points is shown in Fig. 5. The respective position of the grid points on the mapped quantum space with three qubits is shown in Fig. 5 (a). Fig. 5 (b) shows the structure of the MPS representation of the field, depicting the transformation from a dense rank-3 tensor ( $q = 3$  indices each of dimension  $K = 2$ ) to that of a tensor train of three low rank tensors.

The bond dimension is the dimension of the tensor that is used to perform tensor contraction with another tensor, and its value can be different for different tensors. For a full-rank decomposition, the value of  $d$  increases exponentially from the left to the middle of the tensor chain ( $\min(j, q - j)$ ) and then decreases exponentially to the rightmost tensor. Thus, the bond dimension is exponentially large for a full-rank representation of the original function. However, the number of nonzero singular values in the decomposition is usually small compared to the full rank. Thus, we can impose a



**Fig. 5** Depiction of a sample 1D velocity field on a mesh of size  $N = 8$ : (a) mapping of the classical grid points to quantum registers, and (b) representation of the field in a dense tensor form and the structure of the corresponding MPS.

low-rank approximation of the original state by only retaining the  $d$  largest singular values in the decomposition at each bond. With the low-rank approximation, the original function of the  $K^q$  data points is replaced by a series of tensors with  $\sim q(Kd^2)$  elements in total, leading to a significant reduction of data points at the cost of some accuracy. This accuracy can be improved by increasing the bond dimension.

Using the MPS framework, we can adaptively manage the computational resources needed to approximate solutions to the Burgers' equations effectively. This method provides a scalable way to handle complex nonequilibrium phenomena.

## 2. Quantum mapping of linear and nonlinear spatial differential operators

Once the flow variables are embedded in the quantum state, we now map the classical spatial differential operations (linear and nonlinear) to quantum operators. Below, we first elaborate on how the linear term—the velocity diffusion term,  $\nabla^2 u$ —is represented by matrix product operators (MPOs). Then, we show how the nonlinear convective term,  $u \cdot \nabla u$ , can be mapped using a nonlinear MPO.

**Mapping the linear spatial differential operators into MPOs:** Once the MPS state is constructed, spatial differential operators, both first and second orders, can be easily mapped into corresponding MPOs [24]. Using the finite difference method, regardless of the order of accuracy of the differencing scheme, the differentiation of a function  $u(x_i)$  at position  $x_i$  becomes a linear combination of itself and its displacement  $u(x_i \pm n\delta x)$ . Consequently, within the MPS representation, the differentiation operator is connected to the MPO that induces the state transition  $u(x) \rightarrow u(x \pm n\delta x)$ .

Let us consider approximating the first- and second-order spatial derivatives with a central difference scheme.

$$\frac{\partial u(x_i)}{\partial x} = \frac{u(x_i + \delta x) - u(x_i - \delta x)}{2\delta x} + O(\delta x^2) \quad (7)$$

$$\frac{\partial^2 u(x_i)}{\partial x^2} = \frac{u(x_i + \delta x) + u(x_i - \delta x) - 2u(x_i)}{\delta x^2} + O(\delta x^2) \quad (8)$$

The classical cost of applying these on the grid points is  $O(K^q)$ . Using the MPS embedding of the variables, we can use MPOs to implement the displacements in the grid as

$$\frac{\partial |u(\{x_i\})\rangle}{\partial x} \simeq \frac{S^+ - S^-}{2\delta x} |u(\{x_i\})\rangle \quad (9)$$

$$\frac{\partial^2 |u(\{x_i\})\rangle}{\partial x^2} \simeq \frac{S^+ + S^- - 2}{\delta x^2} |u(\{x_i\})\rangle. \quad (10)$$

Here,  $S^\pm$  are ladder operators that enable the operation of state transitions in the quantum circuit

$$S^+ |u(\{x_i\})\rangle = \begin{cases} |u(\{x_{i+1}\})\rangle, & u(\{x_i\}) < K^q \\ 0, & \text{else} \end{cases} \quad (11)$$

$$S^- = (S^+)^\dagger. \quad (12)$$

The MPO implementations can be performed efficiently using a bond dimension of 3. Thus, the MPO implementation gives a computational cost reduction to  $\mathcal{O}(d^3)$ , leading to exponentially faster operations for representations with small bond dimension [24].

We note that for computing the finite difference operations in other dimensions, the ladder operator can also be used to perform displacements in the grid. The displacement step in Eq. 11, instead of being unity, will now depend on  $q$  and the particular dimension  $\mathcal{D}$  where the finite difference operation is performed. Furthermore, the operations can be easily extended to incorporate higher order approximations.

**Mapping the nonlinear differential operator into MPOs.** The critical challenge in solving the Burgers' equation on quantum computers is the treatment of the nonlinear term  $u \cdot \nabla u$  due to the linear nature of quantum state evolution. For a given circuit encoding the MPS  $u(\{x_i\})$ , the element-wise multiplication  $u \nabla u$  can be represented by  $u(\{x_i\})[O^\nabla u](\{x_i\})$ , where  $O^\nabla$  represents the MPO for the gradient operator. Hence, following the concept of quantum nonlinear processing unit (QNPU) proposed in Lubasch et al., which is variational-based, we can make a copy of the  $u(\{x_i\})$  circuit, apply unitaries for the  $O^\nabla$  operators, and then add a set of CNOT gates to obtain the element-wise multiplication between  $u(\{x_i\})$  and  $[O^\nabla u](\{x_i\})$ . Note that our interest in this variational method in the present study is to obtain a reference solution for validating an alternative approach that does not use variational algorithms [20], as described in Appendix A.A. The latter will be implemented in the future.

### 3. Time evolution of the quantum operations

Let us consider the state representation of the velocity in terms of MPS, the initial state,  $|u(\{x_i\}, t_0)\rangle$ , and the Hamiltonian,  $\hat{H}$ , for the time evolution operation determined by the mapping of the spatiotemporal differential operations into MPOs. Now we can represent the time evolution of the states using spectral-step method [25] as

$$|u(\{x_i\}, t)\rangle = e^{i\hat{H}t} |u(\{x_i\}, t_0)\rangle. \quad (13)$$

If the Hamiltonian is a combination of multiple MPOs, we can rely on the Trotter–Suzuki expansion to split the operation and obtain an approximate solution [24]. In discrete time, the above formulation becomes

$$|u(\{x_i\}, t_0 + M)\rangle = \left( e^{i\hat{H}\delta\tau} \right)^M |u(\{x_i\}, t_0)\rangle, \quad (14)$$

where  $\delta\tau$  is the discrete time-step and  $M$  is the total number of time steps to evolve the states. A sample depiction of this is shown in Fig. 4. Given the circuit representation of the Hamiltonian, standard implementation of Trotterization (available in various SDKs) can be used to implement the time evolution of the velocity state. Nonetheless, the circuit depth can increase prohibitively with increase in time steps.

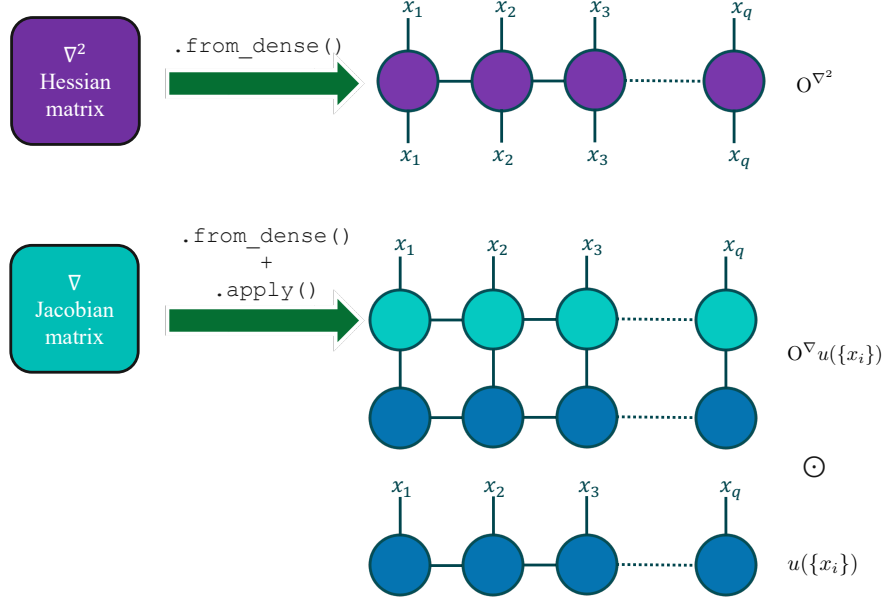
It is worth noting that in this study we evolve the states using the classical Euler method for the sake of a thorough investigation of the tensor network algorithm for the spatial derivatives in the Burgers' equation. Nevertheless, we elaborate an alternative approach using fast-forwarding algorithms [20] in Appendix A.B for future study.

## IV. Current implementation: Verification and validation of the tensor network algorithm

In this preliminary implementation of the algorithm, we focus on verifying and validating the classical tensor network components of the state embedding using MPS (section III.B.1) and the MPO representation of the linear and nonlinear differential operators (III.B.2). For the time evolution operation, we advance the solution in time with the explicit Euler method to validate the functionality and robustness of these tensor network operations.

The algorithm is verified and validated by following two steps. First, we verify individual operators for each physical term, including the linear and nonlinear components in Eq. 2. Second, we apply the explicit Euler method to evolve the states for validating the combined MPO operators and the source component for Eq. 2 compared to the classical HPC solution, as shown in Fig. 1.

We use the `quimb` package (version 1.11.2) [26] to compute the MPS and MPO representations. For the MPO representation of the linear differential operators, we first construct the respective Jacobian and Hessian matrices for the  $\nabla$  and  $\nabla^2$  operations, which use a centered differencing scheme for interior grid points, while boundary schemes are one-sided. Then, the `quimb.tensor.MatrixProductOperator.from_dense()` functionality of the `quimb` package is used to generate MPOs from these dense matrices, as depicted in Fig. 6. For the nonlinear operator,  $u \cdot \nabla$ , we take a Hadamard product between the MPS  $u(\{x_i\})$  and the result of applying the gradient MPO,  $O^\nabla u$ , on  $u(\{x_i\})$ , denoted as  $u(\{x_i\}) \odot [O^\nabla u](\{x_i\})$ .



**Fig. 6** Depiction of constructing MPOs of the  $\nabla^2$  and  $u \cdot \nabla u$  operations using the `quimb` library.

On the construction of quantum circuits, a circuit to represent the MPS can be efficiently achieved using unitary MPOs as elaborated in Ran. Representing the MPOs of the differential operators as quantum circuits is straightforward as long as they are unitary. However, the MPOs generated in our case need not be unitary operations. Thus, the challenge lies in mapping the linear and nonlinear MPOs to unitary operations, which can be done using strategies such as linear combination of unitaries or optimization of a parametrized quantum circuit. We leave the time evolution component of the algorithm and the challenge of quantum circuit construction to future work.

## V. Results and Discussions

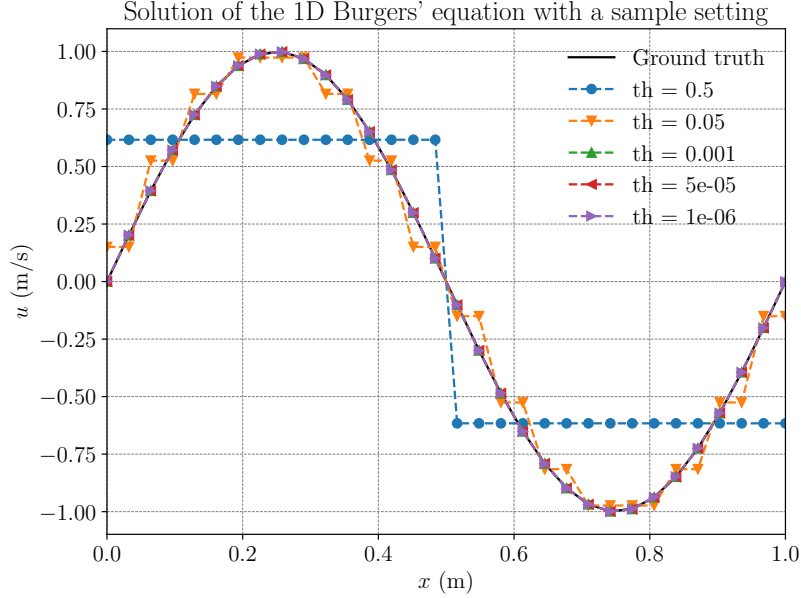
The tensor network algorithm is applied to solve Eq. 2. In the following, we provide results for verifying and validating two components of the algorithm: (1) state embedding of velocity using MPS and (2) mapping the linear and nonlinear differential operators into MPOs. We apply these methods to two test cases: the velocity initialized as a simple sine wave and the complex multimodal form described in Eq. 4.

### A. Verifying MPS embedding of flow state

As described in section III.B.1, the key component for the accuracy and computational complexity of the MPS embedding depends on the number of nonzero singular values retained in the Schmidt decomposition. This can be either controlled by setting a predetermined maximum bond dimension or by imposing a threshold on the singular values if their sparsity is not high. Figure 7 denotes the effect of this threshold on representing the sine wave initial condition for solving Burgers' equation. As the threshold decreases, the accuracy of the MPS embedding increases, retaining more singular values resulting in a higher-order approximation. Figure 8 shows the same analysis for the complex multimodal flow initialization in Walters et al. at two different grid resolutions,  $N = 1024$  (top) and  $N = 4096$  (bottom). We note that the MPS decomposition of this flow initialization converges around the same threshold even with different grid resolutions. Further analysis of the convergence of the decomposition with respect to the threshold is provided in the



next section.



**Fig. 7** Comparing the MPS representation of the sine wave initial condition for the Burgers' equation with change in singular value threshold.

### B. Verifying MPO representation of linear and nonlinear differential operators

Similarly to the MPS decomposition, the MPO representation of the spatial differential operators also depends on the singular values retained. In Fig. 9, we show the convergence of the MPS and MPO (of  $\nabla$ ,  $\nabla^2$ , and  $u \cdot \nabla$  operators) representations of the sine wave initialization with change in threshold at various grid resolutions. Convergence is measured by the maximum absolute difference,  $\max \|\cdot\|_\infty$ , of the reconstructed vector and matrices from MPS and MPOs, respectively. Note that the threshold values are in reverse order on the  $x$ -axis, decreasing towards the right, for better readability.

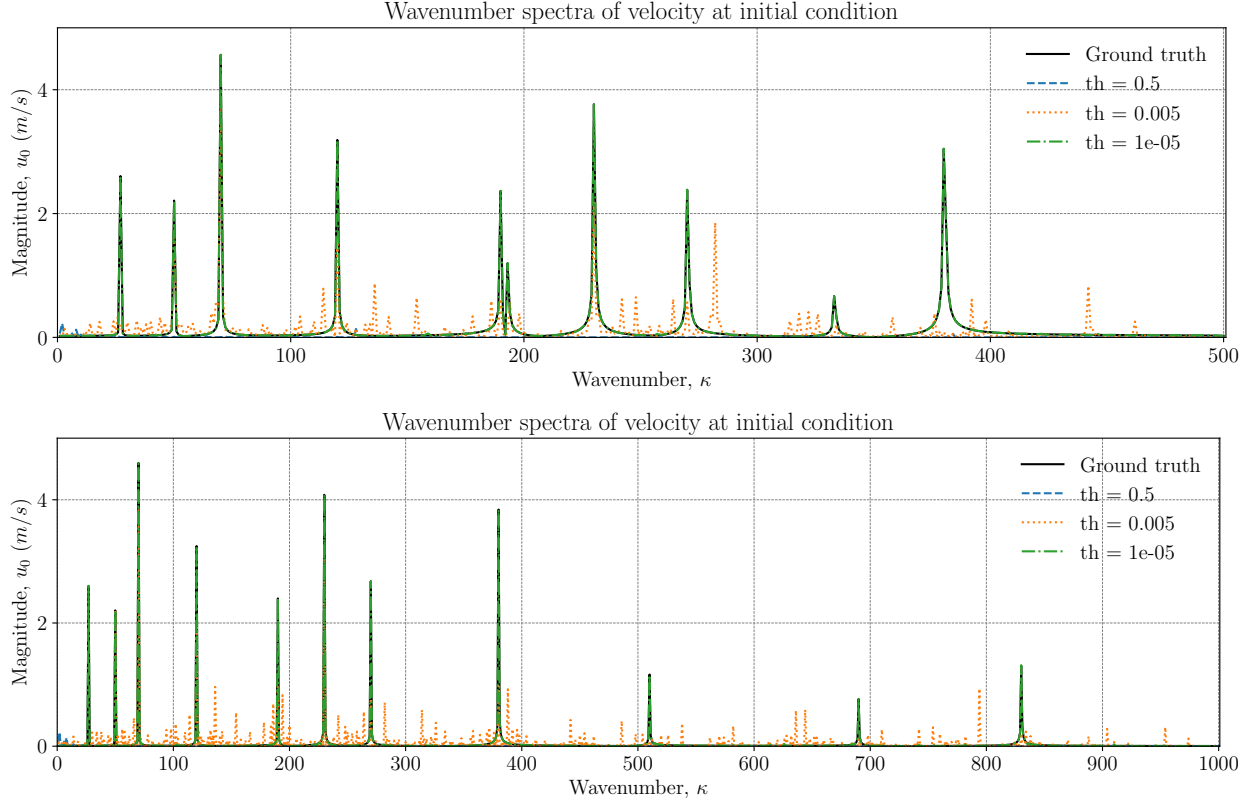
For both the MPS and the MPO representation, the threshold for convergence decreases with grid resolution - a lower value of threshold denotes a more stringent constraint, retaining more singular values and larger bond-dimension. We note that with an increase in grid resolution, the threshold required for convergence of the MPS embedding is lower than that of the linear MPOs. However, the convergence threshold of the nonlinear MPO is dominated by that of the MPS, due to a larger amplitude of  $u$  in the  $u \cdot \nabla$  operation. The converged accuracy of the nonlinear MPO is similar to the least of the MPS and gradient MPO. The converged accuracy of the MPS remains constant with a change in grid resolution, whereas that of the MPOs increases with grid resolution.

In Fig. 10, we show the results for the flow initialized using the multimodal function of Walters et al.. A similar behavior to that of the sine wave initialization is observed. However, the MPS embedding, and so does the nonlinear MPO, at grid resolutions  $N = 4096, 16384$  does not fully converge even with threshold values  $O(10^{-12})$ . We note that the convergence behavior and accuracy of the operators are similar at these resolutions, and thus, to validate the algorithm, we simulate the flow at  $N = 8192$ .

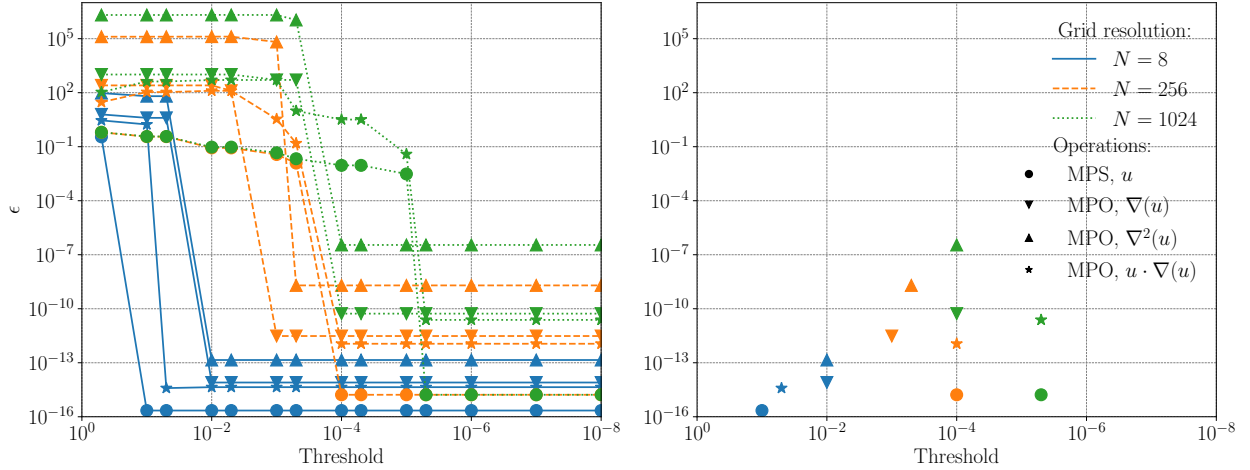
### C. Validation of operators with classical time-marching

We use classical time marching of the full Burgers' equation, Eq. 2, to validate the functionality and test the robustness of the tensor network operations to model the time evolving PDE. We use the explicit Euler method for time-marching, which for the velocity at discrete time  $t + 1$  is given by

$$u(x, t + 1) = u(x, t) + \delta t \left[ \nu \nabla^2 u(x, t) - u(x, t) \cdot \nabla u(x, t) + g(x, t) \right]. \quad (15)$$

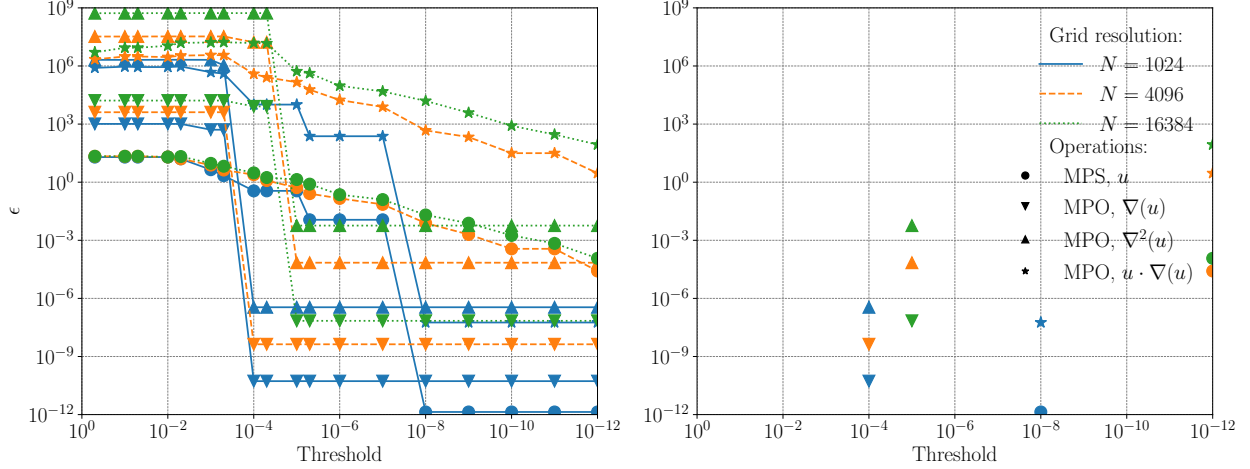


**Fig. 8** Comparing the MPS representation of the flow initialized in Walters et al. for the Burgers' equation with change in singular value threshold at a grid resolution of  $N = 1024$  (top) and  $N = 4096$  (bottom).



**Fig. 9** (Left) Convergence of the MPS and MPO representations with threshold of singular values retained for the sine wave initial condition at various grid resolutions. (Right) The respective converged values for each case. Here, the error metric  $\epsilon$  is defined by the  $\max \|\cdot\|_\infty$  of the reconstructed data from the various operations.

At each time step of the solver, the state vectors from all operations comprise the right-hand side (RHS). The main reason for choosing the explicit Euler method is for a straightforward comparison between the classical RHS and the



**Fig. 10** Convergence of MPS and MPO with threshold of singular values retained for the flow initialized in Walters et al. at various grid resolutions. (Left) The convergence plot and (right) the respective converged solution of each case. The error metric  $\epsilon$  is defined by the  $\max \|\cdot\|_\infty$  of the reconstructed data from the various operations.

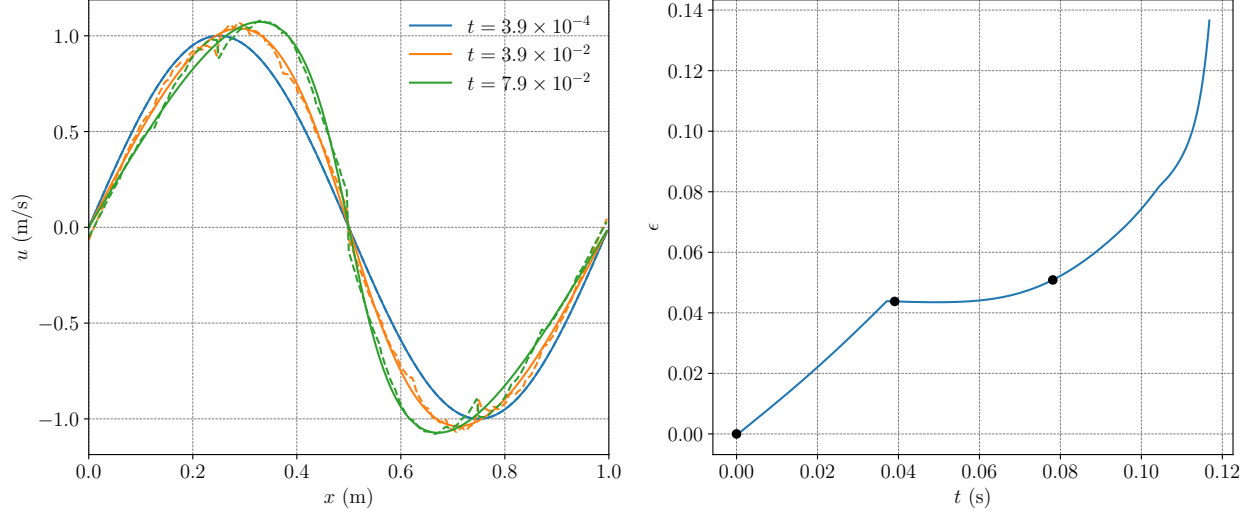
RHS using the tensor network algorithm at each time step. The results of applying the linear and nonlinear MPOs, which return MPSs, are converted back to the respective classical state vector using the `.to_dense()` function of `quimb`. A separate MPS for the source term is created. The sum of all these terms forms the RHS. We note that the explicit Euler method is neither accurate nor stable for longer time, but our focus here is for a short-time simulation of a smooth flow.

The time step,  $\delta t$ , is chosen based on the Courant–Friedrichs–Lewy (CFL) value of 0.1 and 0.05 for the sine wave ( $N = 256$ ) and Walters et al. ( $N = 8192$ ) test cases, respectively. Based on the convergence results in Section V.B, we select a threshold value of  $1 \times 10^{-6}$  and  $1 \times 10^{-5}$  for the sine wave and Walters et al. test cases, respectively. The threshold value is kept constant throughout the simulation.

In Fig. 11, we show the results of the sine wave flow, with the error at each time instant measured using a normalized L2 norm. After the first time step, the tensor network operations still produce accurate results, thus validating their functionality. The error of the tensor network-based implementation increases linearly over an initial time regime with a maximum error of 4%, and then the error increases exponentially. The Euler time marching method may contribute to this error behavior, and we will consider testing it with the standard four-stage Runge-Kutta method.

We show the results of the Walters et al. flow case in Fig. 12. The error being close to 0 in the first time step again validates the functionality of the tensor network algorithm. The error in this complex multimodal case is an order of magnitude higher, but converges with time, around 20%. While larger scale structures are captured accurately over time, there is a limitation of the algorithm in capturing small scale features. Such limitations of tensor network operations to capture the multimodal chaotic behavior in the smaller scales are expected when the number of modes (or singular values) approaches the number of data points and effective compression of the data to MPS (and MPO) is no longer possible. The convergence behavior denotes that the chosen grid resolution  $N = 8192$  would be sufficient for the tensor decompositions to capture the velocity dynamics during the simulated time period.

For both cases, the increase in error with time of the classical time-marching of the tensor network operations lack of full convergence highlights a more fundamental issue: in dynamical problems, the natural spreading of the wavepacket typically necessitates a linear or sub-linear growth in the bond dimension. Consequently, using a fixed threshold (bond dimension) during the evolution inevitably limits the accuracy. On classical computers, however, increasing the bond dimension to retain accuracy leads to an exponential increase in computational cost, posing a major challenge for classical simulation. In contrast, quantum computers possess exponentially large expressive capacity natively, allowing them to avoid such truncation of the bond dimension.



**Fig. 11** Time evolution of the sine wave initial condition using classical and tensor-network based operators. The solid lines are the classical solution,  $u(x, t)$ , and the dashed lines are the solution using the tensor network operators,  $u(\{x_i\}, t)$ . The error in the states is defined by  $\epsilon(t) = \|u(x, t) - u(\{x_i\}, t)\|_2 / \|u(x, t)\|_2$ .

## VI. Concluding Remarks and Future Work

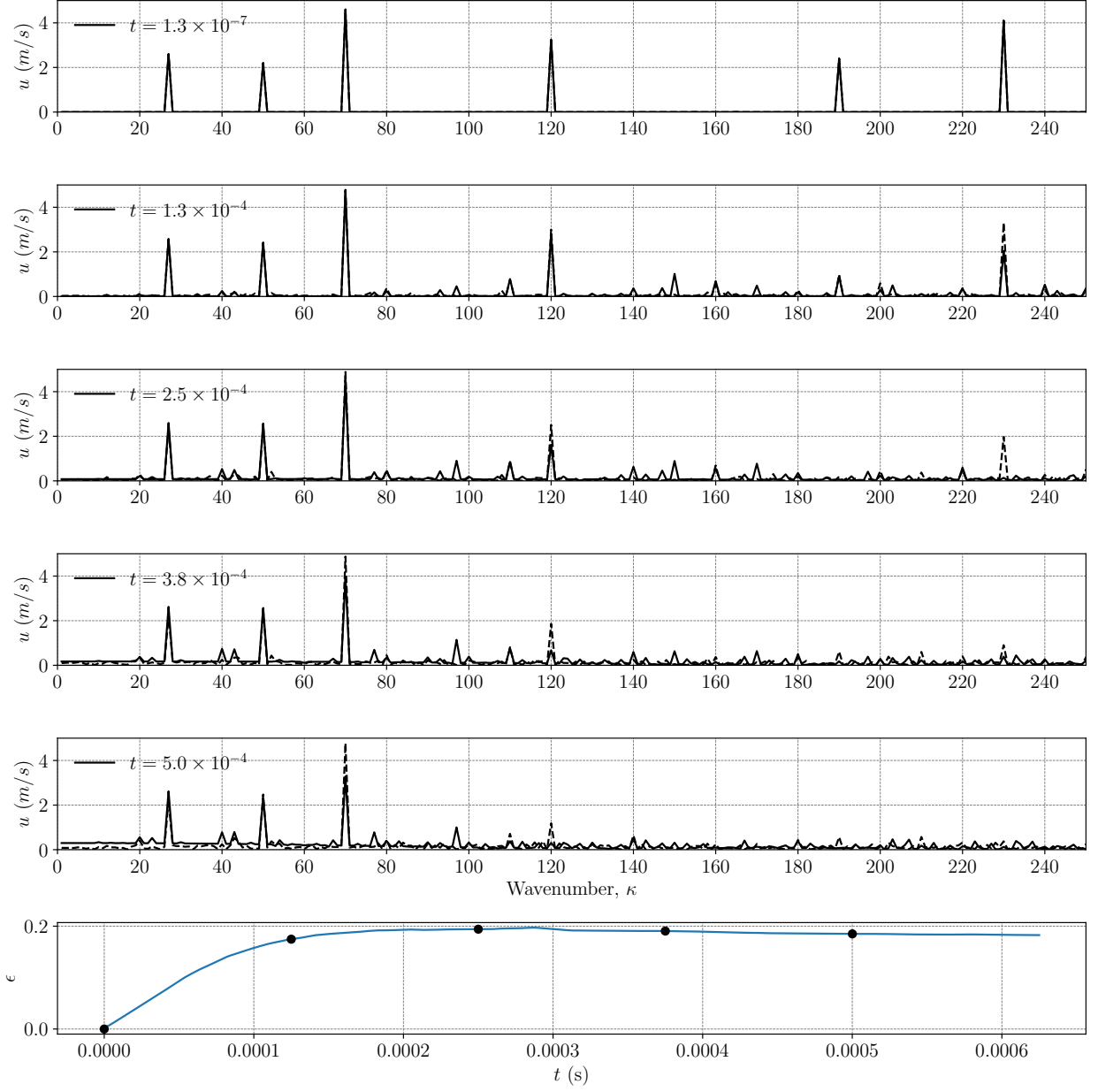
We implement a tensor network-based quantum algorithm to solve the nonlinear terms in PDEs, demonstrated using the 1D Burgers' equation. The quantum algorithm involves the use of tensor network states (matrix product state, MPS) and operations (matrix product operators, MPO) to represent flow variables and the linear and non-linear spatial differential operations. The flow problem is chosen for the straightforward performance comparison between classical HPC solutions and the solutions by the quantum algorithm, in addition to the relevance to fundamental fluid dynamics modeling. In the current work, we focus on the verification and validation of the tensor network operations in the algorithm using Euler time marching method to evolve the solution. Future work will involve implementing quantum time evolution using standard methods such as Trotterization (or Trotterized real time evolution) [28, 29] and time evolving block decimation [30]. We will also implement the conversion of the tensor network operations to actual quantum circuits and benchmark them on quantum simulators and real hardware. Furthermore, we will explore the alternative approaches for nonlinear MPO and time evolution proposed in [20] (and in Appendix A.A and A.B). We note that the fundamental aspect of the current algorithm to formulate quantum operators for linear and nonlinear differential operations opens the avenue to extend the algorithm for multi-dimensional and multi-variate nonlinear PDEs, such as the incompressible and compressible Navier-Stokes equations. These will be demonstrated in future work.

## Acknowledgments

This research used resources of the Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory, which is supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC05-00OR22725. The research was partially supported by the Scott Research Fund in Aerospace at the University of Virginia.

## References

- [1] Norman, M. R., Bader, D. A., Eldred, C., Hannah, W. M., Hillman, B. R., Jones, C. R., Lee, J. M., Leung, L. R., Lyngaas, I., Pressel, K. G., Sreepathi, S., Taylor, M. A., and Yuan, X., "Unprecedented cloud resolution in a GPU-enabled full-physics atmospheric climate simulation on OLCF's summit supercomputer," *International Journal of High Performance Computing Applications*, Vol. 36(1), 2021, pp. 93–105.
- [2] Leyton, S. K., and Osborne, T. J., "A quantum algorithm to solve nonlinear differential equations," *arXiv preprint arXiv:0812.4423*, 2008.
- [3] Yepez, J., "Quantum lattice-gas model for computational fluid dynamics," *Physical Review E*, Vol. 63, No. 4, 2001, p. 046702.



**Fig. 12** Time evolution of the flow initialized in Walters et al. at grid resolution  $N = 8192$  using classical and tensor-network based operators. The solid lines are the classical solution,  $u(x, t)$ , and the dashed lines are the solution using the tensor network operators,  $u(\{x_i\}, t)$ . The error in the states is defined by  $\epsilon(t) = \|u(x, t) - u(\{x_i\}, t)\|_2 / \|u(x, t)\|_2$ .

- [4] Cao, Y., Papageorgiou, A., Petras, I., Traub, J., and Kais, S., “Quantum algorithm and circuit design solving the Poisson equation,” *New Journal of Physics*, Vol. 15, No. 1, 2013, p. 013021.
- [5] Todorova, B. N., and Steijl, R., “Quantum algorithm for the collisionless Boltzmann equation,” *Journal of Computational Physics*, Vol. 409, 2020, p. 109347.
- [6] Budinski, L., “Quantum algorithm for the advection–diffusion equation simulated with the lattice Boltzmann method,” *Quantum Information Processing*, Vol. 20, No. 2, 2021, p. 57.

- [7] Gopalakrishnan Meena, M., Gottiparthi, K. C., Lietz, J. G., Georgiadou, A., and Coello Pérez, E. A., “Solving the Hele–Shaw flow using the Harrow–Hassidim–Lloyd algorithm on superconducting devices: A study of efficiency and challenges,” *Physics of Fluids*, Vol. 36, No. 10, 2024.
- [8] Harrow, A. W., Hassidim, A., and Lloyd, S., “Quantum algorithm for linear systems of equations,” *Physical review letters*, Vol. 103, No. 15, 2009, p. 150502.
- [9] Childs, A. M., Liu, J.-P., and Ostrander, A., “High-precision quantum algorithms for partial differential equations,” *Quantum*, Vol. 5, 2021, p. 574.
- [10] Liu, J.-P., Kolden, H. Ø., Krovi, H. K., Loureiro, N. F., Trivisa, K., and Childs, A. M., “Efficient quantum algorithm for dissipative nonlinear differential equations,” *Proceedings of the National Academy of Sciences*, Vol. 118, No. 35, 2021, p. e2026805118.
- [11] Itani, W., Sreenivasan, K. R., and Succi, S., “Quantum algorithm for lattice Boltzmann (QALB) simulation of incompressible fluids with a nonlinear collision term,” *Physics of Fluids*, Vol. 36, No. 1, 2024.
- [12] Lin, Y. T., Lowrie, R. B., Aslangil, D., Subaşı, Y., and Sornborger, A. T., “Koopman von Neumann mechanics and the Koopman representation: A perspective on solving nonlinear dynamical systems with quantum computers,” *arXiv preprint arXiv:2202.02188*, 2022.
- [13] Giannakis, D., Ourmazd, A., Pfeffer, P., Schumacher, J., and Slawinska, J., “Embedding classical dynamics in a quantum computer,” *Physical Review A*, Vol. 105, No. 5, 2022, p. 052404.
- [14] Joseph, I., “Koopman–von Neumann approach to quantum simulation of nonlinear classical dynamics,” *Phys. Rev. Res.*, Vol. 2, 2020, p. 043102. <https://doi.org/10.1103/PhysRevResearch.2.043102>, URL <https://link.aps.org/doi/10.1103/PhysRevResearch.2.043102>.
- [15] Steijl, R., “Quantum algorithms for nonlinear equations in fluid mechanics,” *Quantum computing and communications*, 2020.
- [16] Gaitan, F., “Finding flows of a Navier–Stokes fluid through quantum computing,” *npj Quantum Information*, Vol. 6, No. 1, 2020, p. 61.
- [17] Oz, F., San, O., and Kara, K., “An efficient quantum partial differential equation solver with chebyshev points,” *Scientific Reports*, Vol. 13, No. 1, 2023, p. 7767.
- [18] Lubasch, M., Joo, J., Moinier, P., Kiffner, M., and Jaksch, D., “Variational quantum algorithms for nonlinear problems,” *Physical Review A*, Vol. 101, No. 1, 2020, p. 010301.
- [19] Pool, A. J., Somoza, A. D., Mc Keever, C., Lubasch, M., and Horstmann, B., “Nonlinear dynamics as a ground-state solution on quantum computers,” *Physical Review Research*, Vol. 6, No. 3, 2024, p. 033257.
- [20] Gopalakrishnan Meena, M., Zhang, Y., Jiang, W., Lin, Y., Günther, S., and Gao, X., “Towards a Quantum Algorithm for the Incompressible Nonlinear Navier-Stokes Equations,” *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1, IEEE, 2024, pp. 662–668.
- [21] Walters, S., Guzik, S., and Gao, X., “Favre-averaged spatiotemporal-filtered large eddy simulation,” *2018 AIAA SciTech Meeting*, AIAA, 2018.
- [22] Schollwöck, U., “The density-matrix renormalization group in the age of matrix product states,” *Annals of physics*, Vol. 326, No. 1, 2011, pp. 96–192.
- [23] Orús, R., “A practical introduction to tensor networks: Matrix product states and projected entangled pair states,” *Annals of physics*, Vol. 349, 2014, pp. 117–158.
- [24] García-Ripoll, J. J., “Quantum-inspired algorithms for multivariate analysis: from interpolation to partial differential equations,” *Quantum*, Vol. 5, 2021, p. 431.
- [25] Weideman, J., and Herbst, B. M., “Split-step methods for the solution of the nonlinear Schrödinger equation,” *SIAM Journal on Numerical Analysis*, Vol. 23, No. 3, 1986, pp. 485–507.
- [26] Gray, J., “quimb: A python package for quantum information and many-body calculations,” *Journal of Open Source Software*, Vol. 3, No. 29, 2018, p. 819.

- [27] Ran, S.-J., “Encoding of matrix product states into quantum circuits of one-and two-qubit gates,” *Physical Review A*, Vol. 101, No. 3, 2020, p. 032310.
- [28] Magnus, W., “On the exponential solution of differential equations for a linear operator,” *Communications on pure and applied mathematics*, Vol. 7, No. 4, 1954, pp. 649–673.
- [29] Hatano, N., and Suzuki, M., “Finding exponential product formulas of higher orders,” *Quantum annealing and other optimization methods*, Springer, 2005, pp. 37–68.
- [30] Vidal, G., “Efficient classical simulation of slightly entangled quantum computations,” *Physical review letters*, Vol. 91, No. 14, 2003, p. 147902.
- [31] Tkachenko, N. V., Cincio, L., Boldyrev, A. I., Tretiak, S., Dub, P. A., and Zhang, Y., “Quantum Davidson algorithm for excited states,” *Quantum Science and Technology*, Vol. 9, No. 3, 2024, p. 035012. <https://doi.org/10.1088/2058-9565/ad3a97>, URL <https://dx.doi.org/10.1088/2058-9565/ad3a97>.
- [32] Cirstoiu, C., Holmes, Z., Iosue, J., Cincio, L., Coles, P. J., and Sornborger, A., “Variational fast forwarding for quantum simulation beyond the coherence time,” *npj Quantum Information*, Vol. 6, No. 1, 2020, p. 82.
- [33] Bittel, L., and Kliesch, M., “Training variational quantum algorithms is NP-hard,” *Physical review letters*, Vol. 127, No. 12, 2021, p. 120502.
- [34] Lanczos, C., “An iteration method for the solution of the eigenvalue problem of linear differential and integral operators,” *Journal of Research of the National Bureau of Standards*, Vol. 45, No. 5, 1950, pp. 255–282.
- [35] Saad, Y., *Iterative methods for sparse linear systems*, SIAM, 2003.

## A. Proposed algorithms

### A. An alternative MPO for spatial nonlinear differential operator

Unlike the variational algorithm proposed in Lubasch et al., our alternative approach involves the use of a linear combination of unitaries (LCU) to solve  $u$  efficiently [20]. The evolution of the momentum equation Eq. 2 with a time step of  $\delta\tau$  can be represented by a functional form  $u(t + \delta\tau) = e^{-\mathcal{L}\delta\tau}u(t)$ , where  $e^{-\mathcal{L}\delta\tau}$  is a nonlinear and non-unitary evolution operator. Here, we map the scaled  $u'(t + \delta\tau) = c^{-1/2}u(t + \delta\tau)$  into unitaries,

$$u'(t + \delta\tau) = c^{-1/2}e^{-\mathcal{L}\delta\tau}u(t) = e^{-i\delta\tau\hat{A}}u(t),$$

where  $\hat{A} = \sum c_i \hat{P}_i$  is Hermitian and expanded as sum of Pauli strings. Hence, the change in  $u'(t)$  after one step is

$$\Delta_0 = \frac{u'(t + \delta\tau) - u(t)}{\delta\tau} = -\frac{1}{2}u \cdot \nabla u + v\nabla^2 u + g.$$

At the same time,  $u'(t + \delta\tau)$  can be approximated from  $u(t)$  from a unitary operator  $e^{-i\delta\tau\hat{A}} \simeq (1 - i\delta\tau\hat{A})$ , leading to  $\Delta = \frac{e^{-i\delta\tau\hat{A}}u(t) - u(t)}{\delta\tau} = -i\hat{A}u(t)$  difference. Consequently, the coefficients  $\mathbf{c} \equiv \{c_i\}$  can be obtained by minimizing the cost function  $C \equiv \|\Delta_0 - \Delta\|$ , which is given by

$$C = \langle \Delta_0 | \Delta_0 \rangle + \sum_i b_i c_i + \sum_{ij} c_i c_j S_{ij}, \quad (16)$$

where  $b_i = i \langle u | \hat{P}_i^\dagger | \Delta_0 \rangle + h.c.$  and  $S_{ij} = \langle u | \hat{P}_i^\dagger \hat{P}_j | u \rangle$ . Hence, the minimum of  $C$  is determined by the solution  $(S^T + S)\mathbf{c} = \mathbf{b}$ . Thus, obtaining the solution to  $\mathbf{c}$ , the evolution driven by the nonlinear term is readily simulated by a unitary evolution via

$$e^{-i\hat{A}\delta\tau} |u\rangle = e^{-i(\sum_i c_i \hat{P}_i)\delta\tau} |u\rangle, \quad (17)$$

which avoids the Carleman linearization [10] of nonlinear equations.

## B. Fast-forwarding algorithms for time evolution

The Trotterization procedure, which involves multiple MPO operations of the linear and nonlinear terms, results in a linear increase in circuit depth with respect to time steps,  $M$ , and computationally expensive and error prone circuits. As a potential strategy to alleviate this linear growth in circuit depth, we propose to use quantum Krylov subspace methods [31] to fast-forward the simulation [20]. A variational approach to reduce the circuit depth is using the variational fast-forwarding (VFF) algorithm [32], which searches for an approximate diagonalization of the time evolution unitaries,

$$(e^{i\hat{H}\delta\tau})^M \simeq WD(\delta\tau)^M W^\dagger = WD(M\delta\tau)W^\dagger, \quad (18)$$

where the unitary with a single time step  $U(\delta\tau) = e^{i\hat{H}\delta\tau}$  is approximated by a diagonal factorization  $WD(\delta\tau)W^\dagger$ , which enables a constant circuit depth with respect to  $M$ . We note that diagonalizing the linear and nonlinear spatial differential operators can pose challenges and require in-depth research. Although the VFF algorithm could provide a constant circuit depth time evolution simulation, variational methods usually suffer from optimization problems due to the complex manifold of the cost function and stochastic errors (due to decoherence of qubits, control error, readout errors, etc.) [33]. Alternatively, the Krylov subspace method [31, 34, 35] can be used to bypass the optimization problem.